44 · 全链路与性能测试报告（1 亿条数据 · Redis 预热 · 浏览器 E2E · k6/Locust）

项	内容
文档 ID	TG-DOC-44
版本	v1.2（数据规模与性能专项）
编制日期	2026-10-26
依据	GB/T 25000.51（质量度量） · ISO/IEC 25010（性能效率） · 招标"技术实力/演示"分项
被测版本	Tiangong-EWA v1.1 + 分析库/缓存增强（commit 见 PROJECT_UPDATES）
结论	功能 135 项全通过；k6/Locust 均 0 失败；P95 ≤ 4 ms；缓存命中率 99.96%

1. 结论摘要

维度	指标	结果	判定
数据规模	分析库总行数 / 体积	100,000,000 行 / 5.03 GiB	达标
数据装载	1 亿条写入 + 建索引	1,027 s + 357 s（约 17 + 6 分钟）	达标
聚合查询	7 天概览（预聚合后）	0.23 ms（预聚合前 173,537 ms）	提升 ≈ 75 万倍
点查	单作业单时间线（1 亿条中点查）	3.26 ms	达标
缓存	Redis L2 预热	211 key / 138.6 KiB / 1.01 s	达标
缓存	压测期间命中率	99.96%（L1 5,194 / L2 38 / miss 2）	达标
功能	单元 + 接口测试	120 项全通过，覆盖率 93%	达标
功能	浏览器 E2E（全端全链路）	13 项全通过（1440×900 + 390×844）	达标
压测 k6	请求数 / RPS / 失败率	16,823 / 256.7 / 0.00%	达标
压测 k6	全链路 P95 / P99	1.53 ms / 2.26 ms	远优于阈值（300/800 ms）
压测 Locust	请求数 / RPS / 失败率	7,757 / 131.3 / 0.00%	达标
压测 Locust	聚合 P95 / P99	4 ms / 6 ms	达标

> 一句话：1 亿条数据不是"能查"，而是"毫秒级可查、可压、可复现"。
> 关键不在硬件，而在两件事——预聚合读模型与多级缓存预热。

2. 测试环境

项	值
主机	Windows 11（DESKTOP-67ALOT0），开发机（非服务器）
Python	3.13.12（项目内 `.venv`）
服务	granian（ASGI）单进程，`--host 127.0.0.1 --port 8200`
业务库	SQLite WAL，`data/tiangong.db`
分析库	SQLite，`data/tiangong_bulk.db`（5.03 GiB，只读连接）
缓存	Redis 兼容服务（fakeredis `TcpFakeServer`，RESP 协议，127.0.0.1:6379）
浏览器	Chromium 153（Playwright 驱动）
压测工具	k6 v0.54.0；Locust（Python，gevent）
磁盘	D: 剩余 5.7 GiB（数据集与索引占用约 5.1 GiB）

关于 Redis 的环境说明（如实披露）：本机无 `redis-server` 二进制、Docker daemon 未运行、WSL 被安全策略禁用。为完成真实链路验证（TCP 连接、TTL、SCAN 前缀失效、命中率），采用 `fakeredis.TcpFakeServer` 提供 RESP 协议的 TCP 服务——`redis-py` 客户端与应用代码零改动。生产替换为真 Redis / 云 Redis 只需改 `TIANGONG_REDIS_URL`（见 docs/09）。

---

3. 数据层：1 亿条怎么建模

3.1 为什么单独建分析库

业务库（作业单/任务/技能/审计）必须小而快——它承载状态机与不变量，写路径要低延迟。1 亿条作业事实（谁在什么时候、用哪个技能、干了多久、成没成）属于典型的分析型负载，单独落在只读分析库，与业务库物理隔离：

```
■■■ tiangong.db■MB ■■          ■■■ tiangong_bulk.db■5 GiB■
■■■ work_orders  ← ■■■              ■■■ wo_fact      15,000,000
■■■ tasks        ← ■■■■             ■■■ task_fact    40,000,000
■■■ episodes     ← ■■■■■            ■■■ episode_fact 35,000,000
■■■ skills       ← ■■■■             ■■■ audit_fact   10,000,000
■■■ audit_log                        ■■■ agg_*■■■■■■■■■
■■■ zones
```

3.2 表结构与体积

表	行数	含义	索引
`wo_fact`	15,000,000	作业单事实（站点/类型/状态/时长/成功）	`created_ts`、`(site, created_ts)`
`task_fact`	40,000,000	任务事实（类型/机器人/技能/重试）	`order_id`、`start_ts`
`episode_fact`	35,000,000	技能执行事实（成功/时长）	`ts`、`(skill, ts)`
`audit_fact`	10,000,000	审计动作（审批/停机/红线拦截）	`ts`
合计	100,000,000	—	7 个索引

紧凑设计：全部字段为 INTEGER 编码（kind/status/skill 用整数码，语义在 `analytics.py` 还原），单行 25 – 40 字节——这是 1 亿条能压进 5 GiB 的原因。若用 TEXT 主键 + JSON 参数，同样数据量会膨胀 3 – 5 倍。

3.3 装载实测

阶段	耗时	说明
生成 + 写入 1 亿条	1,027 s（约 9.7 万行/s）	每批 10 万行 `executemany`，`journal_mode=OFF`、`synchronous=OFF`
建 7 个索引	357 s	装载后建索引（比边写边维护快得多）
落盘体积	5.03 GiB	含索引与 WAL

> 纪律：装载期关闭日志是为了速度，仅在离线灌数时允许；装载完成后立即切回 WAL（脚本自动执行）。

4. 查询路径：一次"慢查询"引出的架构修正

4.1 问题：索引不等于快

第一版 `analytics.py` 直接对明细表做时间窗聚合（`WHERE created_ts >= ?`）。按常识这应该走 `idx_wo_ts`，实测结果却是：

查询	耗时（1 亿条明细，走索引）
近 1 天概览	61.6 s
近 7 天概览	173.5 s
`COUNT(*)`（scale）	2.16 s

根因：二级索引只存 `(created_ts, rowid)`。7 天窗口命中 116 万行，每行都要按 rowid 回表取 `status/kind/duration_ms`——116 万次随机读，在 5 GiB 文件上就是典型的海量随机 I/O。索引把"全表顺序扫描"换成了"百万次随机读"，反而更慢。

4.2 解法：预聚合读模型（rollup）

聚合口径是固定的（按天/小时/类型/技能/动作），完全可以离线算好——
这就是数仓的物化视图/宽表思路：把随机 I/O 换成一次顺序扫描。

预聚合表	行数	口径
`agg_daily`	1,365	天 × 基地 × 类型：单量/完成/失败/中止/时长和
`agg_daily_task`	91	天：任务数、重试数（任务表无 site 维度，按天单行避免重复计数）
`agg_daily_skill`	728	天 × 技能：episode 数/成功数/时长和 + 6 档时长分桶（用于 p95 估算）
`agg_daily_action`	910	天 × 审计动作
`agg_hourly`	2,161	小时：作业单吞吐

构建耗时 3 分 13 秒（一次全表扫描 + 分组，无随机回表）：

步骤	耗时
`agg_daily`	26.96 s

步骤	耗时
`agg_daily_task`	67.99 s
`agg_daily_skill`	76.04 s
`agg_daily_action`	14.99 s
`agg_hourly`	5.18 s

4.3 效果对比

查询	明细扫描（索引）	预聚合	提升
近 1 天概览	61,641 ms	0.30 ms	205,470×
近 7 天概览	173,537 ms	0.23 ms	754,509×
近 30 天概览	—（预计 > 600 s）	0.46 ms	—
近 90 天（全量）	—	1.06 ms	—
小时吞吐	—	0.05 ms	—
审计分布 7 天	—	0.06 ms	—
单作业单时间线（点查）	—	3.26 ms	明细点查本就该快
数据规模 `COUNT(*)`	2,160 ms	2,646 ms	保持明细（缓存 30 s）

> 说明：`summary` 响应里带 `"source": "rollup" | "raw"`，无预聚合的小库（测试/E2E）自动回退明细扫描，保证同一份代码在 20 万行与 1 亿行两种规模下都正确。

4.4 p95 的口径（诚实标注）

预聚合表不保留原始分布，p95 由时长分桶累计估算：`<1s / 1–5s / 5–30s / 30–60s / 60–120s / ≥120s`，取累计到 95% 的桶上界。响应中标注 `"p95_basis": "bucket_estimate"`。
合成数据的时长是均匀分布（0.5 – 240 s），因此 p95 落在 `≥120s` 桶，返回 240,000 ms——这是口径而非缺陷；真实产线数据分布集中，分桶精度更高（可按需细化为 12 档）。

---

5. 缓存层：Redis 预热与命中

5.1 实现

项	设计
L1	进程内 TTL 字典，命中路径无 I/O
L2	`RedisL2`（redis-py），JSON 序列化，前缀 `tg:`，SCAN 前缀失效（不用 KEYS）
降级	L2 任何异常被吞掉 → 只用 L1，功能不受影响（宪法第四条降级纪律）
观测	`hits / misses / l1_hits / l2_hits / hit_rate / l2_available` 暴露于 `/observability/metrics`
安全	红线判定与人工令牌永不进缓存（缓存会引入时序漏洞）

5.2 预热

```
python tools/redis_compat.py                # ■ Redis ■■■■
python tools/warmup_cache.py --bulk-db data/tiangong_bulk.db --days 1,7,30,90 --timelines 200
```

项	结果
写入 key	211（scale 1 + summary 4 + hourly 2 + audit 4 + timeline 200）
体积	138.6 KiB
耗时	1.01 s
TTL	600 s + 0 – 60 s 随机抖动（避免同时过期造成雪崩）
校验	`tg:an:scale` / `tg:an:summary:7` / `tg:an:timeline:15000000` 均命中，TTL 607/614/610 s

5.3 预热效果（冷进程首请求）

场景	延迟
进程刚启动，首个 `summary?days=7`（L2 命中）	19.6 ms
第二次（L1 命中）	2.5 ms
第三次（L1 命中）	1.9 ms
未预热的 `summary?days=60`（rollup 计算 + 回填）	4.7 ms

5.4 压测期缓存表现

指标	值
命中	5,232
未命中	2
命中率	99.96%
L1 / L2 命中	5,194 / 38

5.5 过程中发现并修复的缺陷（重要）

缺陷：`MultiLevelCache.get()` 在 L2 命中后调用 `self.set(key, v)` 回填 L1，而 `set()` 会用默认 TTL（5 s）同时重写 L2——于是"预热 600 s 的键"被一次读操作 降级成 5 s，随后集体过期，预热形同虚设。

现象：预热完成后立刻查询，`tg:an:scale`、`tg:an:summary:7` 的 TTL 变成 -2（已过期），而未被读取的 `tg:an:summary:30` 仍为 600 s。

修复：新增 `_set_l1()`，L2 命中只回填 L1，不改写 L2 的 TTL；并补回归测试 `test_l2_hit_does_not_shorten_l2_ttl`（断言 L2 TTL 仍 > 500 s）。

> 这类缺陷只在"预热 + 长 TTL"组合下暴露，单测不覆盖就一定会漏。已在 docs/08 记录为测试用例。

---

6. 功能测试（全端全链路）

6.1 层次与数量

层次	工具	数量	结果
领域核单测	pytest	44	通过
编排/接口/MCP	pytest + Litestar TestClient	14	通过

层次	工具	数量	结果
v1.1 基础设施（缓存/事件/追踪/仓储/适配器/记忆/A2A）	pytest	38	通过
分析库与 Redis L2	pytest + fakeredis	24（新增）	通过
单元 + 接口合计	—	120	全通过，覆盖率 93%
浏览器 E2E	Playwright + Chromium	13（新增）	全通过

6.2 E2E 用例清单（真实浏览器 + 真实服务进程）

#	用例	断言
1	首屏实时渲染	数据源徽标 = "实时 API"，控制台零错误
2	机群表与 API 一致	行数 == `/robots` 条数，KPI 在线数一致
3	安全区地图	区域矩形数 == `/zones`，机位圆点数 == 机器人 数
4	技能与适配器	行数与 API 一致
5	KPI 与 API 对账	分析库行数、7 天单量逐一比对（防"看板好看但对不上账"）
6	系统指标卡	P95、命中率有值
7	下单 → 审批 → 执行	状态 PENDING→APPROVED→DONE，证据 ≥ 3 条，服务端状态一致
8	红线 R1	目标设为禁入区（55,55）→ ABORTED，提示含 "R1"
9	红线 R2	重量 80 kg 无令牌 → ABORTED，提示含 "R2"
10	移动端 390×844	无横向滚动（溢出 ≤ 1 px）
11	接口不可达	徽标 = "演示兜底"，页面仍有 4 行机群数据，提示横幅可见
12	刷新按钮	刷新后仍为实时数据源
13	截图留证	桌面/移动/下单完成三张截图归档

测试夹具：`tests/e2e/conftest.py` 会拉起独立 granian 进程（临时业务库 + 20 万行分析库 + 空闲端口），用真实 HTTP 与真实浏览器，不是 mock。默认 `pytest` 用 `-m "not e2e"` 跳过浏览器用例，`make e2e` 单独执行。

6.3 E2E 过程中修复的问题

#	问题	修复
1	看板为静态演示数据，无法验证"端到端"	改造为实时数据源（API 驱动 + 兜底），新增下单/审批/执行表单
2	`file://` 打开跨域被拦	服务端同源托管 `web/`（`/` 返回看板）+ CORS

#	问题	修复
3	电量显示 20 位小数	`Math.round()`
4	适配器 dry-run 显示为 down（误导）	区分 `ok` / `■■■(dry-run)` / `down`
5	任务数列恒为 "—"	`/work-orders` 补 `tasks` 计数
6	移动端横向溢出 22 px	长串 `overflow-wrap:anywhere`
7	控制台 404（favicon）	内联 `data:` 图标

---

7. 压力与性能测试

7.1 场景设计（两个工具同口径）

场景	并发模型	行为
`read_heavy`（k6）	5 → 20 VU 阶梯，65 s	机群/作业单/技能/指标/7 天聚合（缓存命中），每 5 轮打一次 60 天窗口（未预热）
`write_chain`（k6）	4 VU 定压，60 s	下单 → 审批 → 执行（含安全审查、证据落库、事件发布）
Locust 混合	30 用户，5/s 爬坡，60 s	6:分析读 / 4:机群读 / 2:写链路 / 1:审计+适配器

阈值（写进脚本，不达标即失败）：`http_req_failed < 1%`；read P95 < 300 ms；write P95 < 1200 ms；缓存聚合 P95 < 120 ms；链路失败率 < 2%。

7.2 k6 结果

指标	值	阈值	判定
请求总数	16,823	—	—
吞吐	256.7 req/s	—	—
迭代	3,370（51.4 it/s）	—	—
失败率	0.00%（0 / 16,823）	< 1%	
检查通过	20,193 / 20,193（100%）	—	
全链路 P95 / P99	1.53 ms / 2.26 ms	300 / 800 ms	
read_heavy P95 / P99	1.46 ms / 2.00 ms	< 300 ms	
write_chain P95 / P99	2.66 ms / 6.13 ms	< 1200 ms	
下单闭环 P95 / P99	7 ms / 14 ms	—	
缓存聚合 P95	1 ms	< 120 ms	
未预热 60 天 P95	1 ms	—	（rollup）
最大单请求	90.66 ms	—	冷启动抖动
传输量	31 MB 收 / 1.9 MB 发	—	—

7.3 Locust 结果（交叉验证）

端点	请求数	失败	中位数	P95	最大	
`GET /analytics/summary?days=[7\	30]`	1,564	0	2 ms	4 ms	88 ms
`GET /robots`	1,060	0	2 ms	4 ms	8 ms	
`GET /work-orders`	1,060	0	1 ms	3 ms	7 ms	
`GET /metrics`	1,060	0	3 ms	5 ms	17 ms	
`GET /skills`	1,060	0	1 ms	2 ms	6 ms	
`POST /work-orders`（下单）	485	0	3 ms	5 ms	93 ms	
`POST .../approve`	485	0	2 ms	3 ms	7 ms	
`POST .../run`（执行）	485	0	2 ms	6 ms	110 ms	
聚合	7,757	0（0.00%）	2 ms	4 ms	110 ms	

吞吐 131.3 req/s（30 用户、含思考时间），失败率 0.00%。

7.4 两工具差异说明

k6 吞吐（256.7 rps）高于 Locust（131.3 rps），原因不是服务端能力差异，而是负载模型不同： k6 的 read 场景思考时间 0.2 – 0.5 s 且写场景独立并发；Locust 30 用户共享 0.2 – 0.8 s 思考时间且写链路占 2/12 权重。 两者 P95 均 ≤ 4 ms，结论一致。

7.5 服务端压测后指标

指标	值
span 总数 / 错误	4,848 / 0
编排 P95（span 级）	0.018 ms
事件发布 / 失败	4,848 / 0
缓存命中率	99.96%
episode 入库（压测产生）	2,424（transport 808 / pick_place 1,616，成功率 100%）

---

8. 瓶颈分析与调优记录

#	现象	根因	处置	效果
1	7 天聚合 173 s	二级索引回表 → 百万次随机读	预聚合读模型	0.23 ms
2	预热键几分钟后失效	L2 命中回填 L1 时连带重写 L2 短 TTL	拆分 `_set_l1`，不改 L2 TTL	命中率 99.96%
3	`COUNT(*)` 2.6 s	1 亿行全表计数	缓存 30 s + 只在看板展示	可接受

#	现象	根因	处置	效果
4	任务数聚合虚高 15 倍	任务表无 site 维度，按 (day,site,kind) 存重复计数	独立 `agg_daily_task` 按天单行	口径正确
5	冷启动首屏 19.6 ms	进程内 L1 为空	Redis 预热 + L2	次请求 1.9 ms
6	单进程写库	SQLite 单写者	压测单实例；扩展路径见 docs/09（PostgreSQL）	写 P95 2.66 ms

已知边界（不粉饰）：

- SQLite 单写者：写吞吐上限受限于单连接串行写；生产规模化需 PostgreSQL（A7/A9 决策）。
- 单进程 Python：GIL 下 CPU 密集查询会互相影响；本次压测读路径几乎全命中缓存，未触顶。
- `fakeredis` 非生产级：单机内存实现，无持久化/集群；仅用于链路验证。
- 合成数据分布均匀：真实产线时长/成功率分布更集中，p95 分桶估算会更准。
- 压测在本机（与压测工具同机），存在资源竞争；数值偏保守。

---

9. 复现步骤

```
# 0. ■■■■■■■■■■
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r impl/services/agent-runtime/requirements.txt
.venv/Scripts/python.exe -m pip install redis fakeredis locust playwright

# 1. ■■ 1 ■■■■■■■■ 17 ■■ + ■■■■ 6 ■■■■
python tools/gen_bulk_data.py --rows 100000000 --db data/tiangong_bulk.db --force

# 2. ■■■■■■■■■■■■ 3 ■■■■
python tools/build_rollup.py --db data/tiangong_bulk.db --rebuild

# 3. ■■■■■■■
python tools/redis_compat.py &
python tools/warmup_cache.py --bulk-db data/tiangong_bulk.db --days 1,7,30,90 --timelines 200

# 4. ■■■■■■■■■■ + ■■■■ + Redis L2■
cd impl/services/agent-runtime
TIANGONG_DB=../../data/tiangong.db \
TIANGONG_BULK_DB=../../data/tiangong_bulk.db \
TIANGONG_REDIS_URL=redis://127.0.0.1:6379/0 \
PYTHONPATH="../../packages/tiangong_core;." \
../../../.venv/Scripts/python.exe -m granian --interface asgi app.server:app --port 8200

# 5. ■■■■■
cd impl && make test && make e2e

# 6. ■■
make perf-k6
make perf-locust
```

---

10. 附件与产物

类型	路径
k6 原始汇总	`impl/tests/perf/results/k6-summary.json`
Locust 统计	`impl/tests/perf/results/locust_stats.csv`、`locust_stats_history.csv`
Locust 失败/异常	`locust_failures.csv`、`locust_exceptions.csv`（均为空）

类型	路径
看板截图	`impl/tests/e2e/artifacts/dashboard-desktop.png`、`dashboard-mobile.png`、`order-done.png`
压测脚本	`impl/tests/perf/k6_tiangong.js`、`impl/tests/perf/locustfile.py`
数据脚本	`tools/gen_bulk_data.py`、`tools/build_rollup.py`、`tools/warmup_cache.py`、`tools/redis_compat.py`

---

11. 对文档体系的回写

文档	变更
docs/03	新增"分析库与读模型"（CQRS 的读侧延伸）、预聚合与缓存预热架构决策 A14/A15
docs/08	新增性能测试层（k6/Locust）、E2E 用例清单、测试数量 120+13
docs/09	新增压测环境装配（`TIANGONG_BULK_DB`/`TIANGONG_REDIS_URL`）与容量基线
docs/38	巡检项新增"预聚合新鲜度""缓存命中率""分析库可用"
README	徽标与"五分钟看全貌"补充数据规模与压测结论